



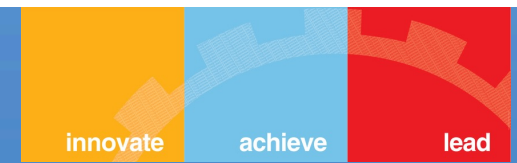
BITS Pilani

Conversational AI

Prompting and Finetuning

Dr Bharathi R

Adjunct Professor, CSIS WILP BITS PILANI



BITS Pilani



AIMLCZG521 , Conversational AI

Lecture No. 6 | Module 2

Disclaimer & Acknowledgement



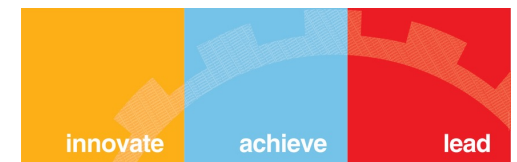
DISCLAIMER

These instructional materials have been developed by the instructor for educational purposes. All content is intended solely to support the learning objectives of this course. While every effort has been made to ensure accuracy and completeness, learners are encouraged to consult original source publications for authoritative reference.

ACKNOWLEDGEMENT

These instructional materials have been developed by the instructor with due acknowledgement of the original publication authors and the broader academic community for their openly accessible resources, along with special recognition of Prof. Bhagath's Conv-AI-WILP (October 2025) presentation materials, which have been adapted and modified to support the learning objectives of this course.

Session Outline



1. Part 1: Prompt Engineering Techniques
2. Part 2: Instruction Tuning & Self-Instruct
3. Part 4: Summary & Next Steps

1. Prompt Engineering



What is Prompt Engineering?

The art and science of designing input prompts to guide LLMs toward desired outputs.

Key Focus:

- Instruction clarity
- Context provision
- Output format specification
- Few-shot examples
- Chain-of-thought reasoning

Why It Matters

- ✓ Immediate deployment
- ✓ No model modification
- ✓ Works with any LLM
- ✓ Cost-effective
- ✓ Enables zero-shot learning

Challenges:

- ✗ Requires experimentation
- ✗ Hard to scale
- ✗ Limited by model capacity



Prompt structure Framework

Effective prompts follow a consistent structure for clarity and control:

- **Priming:** This clearly defines the role of the LLM. (Assign a Persona)
- **Style and Tone:** It sets expectations for clear, concise, and jargon-free language with a sought tone.
- **Error Handling:** This guides the LLM to politely decline requests outside its defined scope.
- **Content (static or dynamic or both):** Query, examples, data, etc.
- **Output Formatting:** The format ensures the response can be easily used by an application or displayed in a structured way.

The goal is to ensure **Clarity, Consistency, and Control**. Without structure, LLMs can be unpredictable. With structure, they become reliable tools.

Prompt Engineering: Real Example



Detailed prompt:

You are a financial analyst specializing in summarizing quarterly earnings reports for technology companies.

Use clear and concise language, avoiding financial jargon. Explain complex financial concepts in a way that is easy for a non-expert to understand. Maintain a formal and objective tone.

If a request is unrelated to summarizing quarterly earnings reports or providing information about them, politely decline and state that it falls outside your area of expertise.

User inquiry:

“Can you summarize the main points of Apple’s Q3 2024 earnings report and explain its significance?”

Zero-Shot vs. Few-Shot Prompting



Zero-Shot

No examples provided

Prompt:

'Classify sentiment: "Great product!"'

- ✓ Simple & quick
- ✓ No data needed
- ✗ Lower accuracy

Accuracy: ~70-75%

Use case: Simple tasks

Few-Shot

2-5 examples provided

Prompt:

"Great!" → Positive

"Awful" → Negative

"Great product!" → ?'

- ✓ Higher accuracy
- ✓ Context learning
- ✗ Requires examples

Accuracy: ~85-90%

Use case: Complex tasks

1 Translate English to French:

2 cheese =>

Translate English to French:

sea otter => loutre de mer

peppermint => menthe poivrée

plush girafe => girafe peluche

cheese =>

Chain-of-Thought (CoT) Prompting



Encourage the LLM to “**think step by step**” by providing examples where the reasoning process is explicitly laid out. This is especially useful for tasks requiring **arithmetic, commonsense, or symbolic reasoning**.

Chain-of-thought prompting

Standard Prompting

Model Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The answer is 27. ❌

Chain-of-Thought Prompting

Model Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. $5 + 6 = 11$. The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The cafeteria had 23 apples originally. They used 20 to make lunch. So they had $23 - 20 = 3$. They bought 6 more apples, so they have $3 + 6 = 9$. The answer is 9. ✅

[[Wei et al., 2022](#); also see [Nye et al., 2021](#)]

Tree-of-Thought (ToT) Prompting



An LLM explores **multiple reasoning paths (thoughts)** as a tree. It **self-evaluates intermediate thoughts** and uses search algorithms (such as **BFS** or **DFS**) to decide which path to explore further, enabling more **deliberate problem-solving**.

Example:

""

I'm trying to solve [complex problem/question].
Before giving me a final answer, please explore at least three
different potential approaches or lines of reasoning.
For each approach, briefly explain it, analyze its pros and cons
(or evaluate its likelihood of success/validity).
Then, based on this exploration, provide your final recommended
solution/answer.

""

*Yao, S., et al. (2023). Tree of Thoughts: Deliberate Problem Solving with Large Language Models.
arXiv:2305.10601*

ToT Example: Game of 24



Problem: Make 24 using {4, 9, 10, 13}

Step 1: Decomposition (Generate Thought Candidates)

- Thought A: $13 - 9 = 4 \rightarrow$ Remaining: {4, 4, 10}
- Thought B: $10 - 4 = 6 \rightarrow$ Remaining: {6, 9, 13}
- Thought C: $4 + 9 = 13 \rightarrow$ Remaining: {13, 10, 13}

Step 2: Self-Evaluation (Score Each Path)

- A: $10+4+4=18 \times$ | $4 \times 4 + 10 = 26 \times$ (Unlikely)
- B: Remaining {6, 9, 13}—possible but difficult
- C: {13, 10, 13}—hard to resolve (Likely bad)

Step 3: Pruning & Search (Discard weak branches, expand promising ones)

ToT: Expanding the Best Path



Continuing from Thought B: {6, 9, 13}

The model expands the most promising branch further with new thought candidates:

- B1: $13 - 9 = 4 \rightarrow$ Remaining: {6, 4} $\rightarrow 6 \times 4 = 24 \checkmark$ SUCCESS!
- B2: $13 + 9 = 22 \rightarrow$ Remaining: {6, 22} $\rightarrow$ Can't easily reach 24
- B3: $9 - 6 = 3 \rightarrow$ Remaining: {3, 13} $\rightarrow 13 \times 3 = 39$ (too high)

Solution Found: $(13 - 9) \times 4 + 6 = 4 \times 4 + 6 = 22...$ [correct path]

Key Insight: By exploring and pruning inefficient paths early, ToT finds solutions faster than linear reasoning.

Meta-Prompting: The Model Writes Its Own Prompts



Automatic Prompt Engineer (APE) — introduced by Zhou et al. (2022), arXiv:2211.01910 — turns the LLM on itself. Instead of a human writing the best prompt for a target task, an 'APE LLM' is asked to generate and evaluate candidate prompts, which are then executed by a 'target LLM'. The best-performing prompt is selected automatically.

The APE Pipeline

1. Provide the APE LLM with demonstrations (input→output examples) of the target task.
2. Instruct the APE LLM to generate N candidate prompt instructions (e.g., N=10–50).
3. Execute each candidate prompt on a validation set using the target LLM.
4. Score each prompt by accuracy (or another metric) on the validation set.
5. Return the highest-scoring prompt for use in production.

Step-1: Instruction Generation

A language model is given a task and a few examples (input–output pairs). It generates multiple **candidate prompts or instructions**.



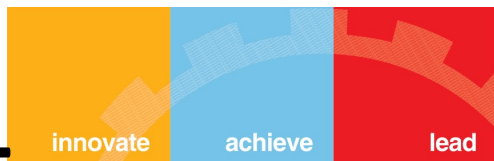
Step-2: Prompt Evaluation

These prompts are tested by running them through a **target model**. The outputs are scored based on **performance metrics** (e.g., accuracy, relevance, coherence).



Step-3: Selection

The **best-performing prompt** is selected for use.



Guidelines for the APE Meta-Prompt

- Clearly define the APE's meta-role: 'You are an expert prompt engineer.'
- Describe the target task in detail with 5–10 high-quality demonstrations.
- Specify what makes an effective prompt: clarity, specificity, correct output format.
- Ask for diverse candidates: 'Generate 10 different prompt instructions, each with a distinct angle.'
- Encourage iteration: 'Refine your best candidate based on the following failure cases.'

SOTA (2024)

APE-discovered prompts matched or outperformed human-written prompts on 24 of 24 NLP tasks evaluated by Zhou et al.

Demonstrations → Automatic Prompt Engineer (APE) → Optimized Prompt

Reference: Zhou, D., et al. (2022): **Large Language Models Are Human-Level Prompt Engineers.**

arXiv preprint: [arXiv:2211.01910](https://arxiv.org/abs/2211.01910)

Decision Framework — Choosing the Right Technique



Technique	Best For	Complexity	Accuracy Gain	Concrete Use Cases
Zero-Shot	Simple tasks	Low	Baseline	Greeting classification, translation, FAQ
Few-Shot	Domain tasks	Low	+10–20%	Product review sentiment, entity extraction
Chain-of-Thought	Reasoning tasks	Medium	+30–50%	Math word problems, legal analysis, debugging
Tree-of-Thought	Complex problems	High	+40–70%	Game of 24, creative writing, proof generation
APE	Unknown/novel tasks	Medium	+15–40%	Any task where optimal phrasing is unclear

💡 **Key Takeaway:** No single technique is always best. Match technique to task complexity and available context.



Part 2: Fine-Tuning & Instruction Tuning

Beyond Prompting

Adapting Models to Your Specific Needs

Language Modeling $\neq$ Following Instructions



Pre-trained LLMs are optimized for next-token prediction, not for following user instructions or maintaining alignment with human intent.

Problem Example:

Prompt: "Explain the moon landing to a 6-year-old."

Completion (unconstrained): "Explain gravity to a 6-year-old... Explain relativity... Explain the big bang..."

The model continues the pattern rather than answering the question. It needs fine-tuning to follow instructions reliably.

Ouyang, L., et al. (2022). Training language models to follow instructions with human feedback. NeurIPS 2022.

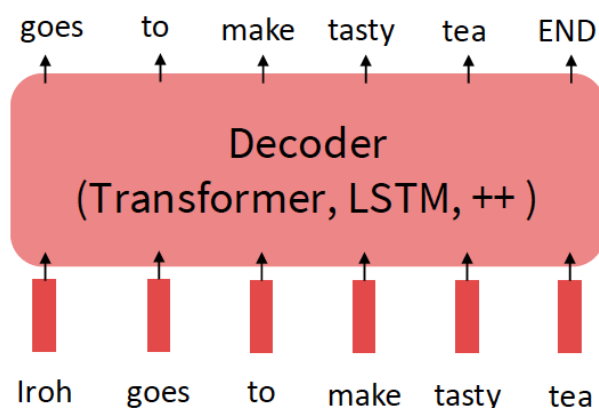
The pretraining/finetuning paradigm



Pretraining can improve NLP applications by serving as parameter initialization.

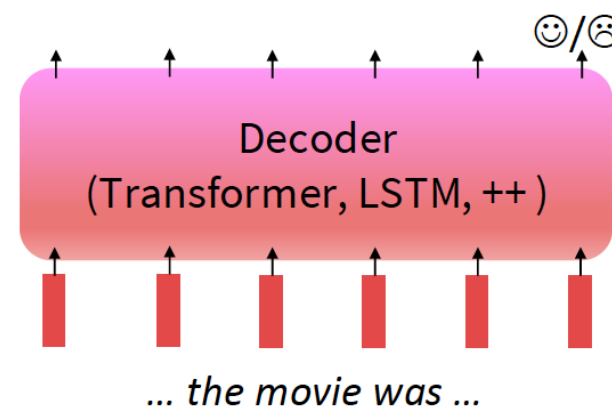
Step 1: Pretrain (on language modeling)

Lots of text; learn general things!



Step 2: Finetune (on your task)

Not many labels; adapt to the task!



- Start with pretrained weights from foundational models
- Fine-tune on smaller, task-specific datasets
- Dramatically reduces data and compute requirements

Result: Better performance, faster training, lower costs

Instruction Tuning



Instruction Tuning

Fine-tuning a pre-trained LLM on a diverse dataset of (instruction, input, output) tuples.

Goal:

- Enable zero-shot generalization
- Improve task performance
- Make models instruction-following

Result: Base model → Instruction-tuned model

Key Differences from Traditional Fine-Tuning

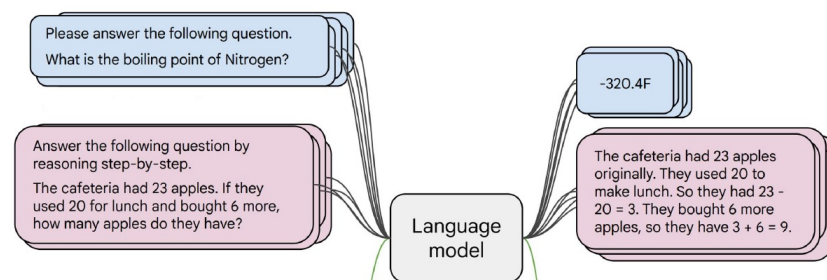
Traditional FT:

- ✗ Single task
- ✗ Task-specific data
- ✗ Poor generalization

Instruction Tuning:

- ✓ Multiple tasks
- ✓ Diverse instructions
- ✓ Excellent generalization
- ✓ Zero-shot transfer

- **Collect examples** of (instruction, output) pairs across many tasks and finetune an LM



- Evaluate on **unseen tasks**

Q: Can Geoffrey Hinton have a conversation with George Washington?
Give the rationale before answering.

Geoffrey Hinton is a British-Canadian computer scientist born in 1947. George Washington died in 1799. Thus, they could not have had a conversation together. So the answer is "no".

[FLAN-T5; [Chung et al., 2022](#)]

Introduction



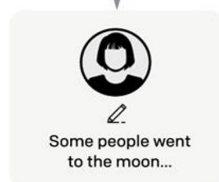
Step 1

Collect demonstration data, and train a supervised policy.

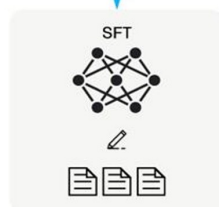
A prompt is sampled from our prompt dataset.



A labeler demonstrates the desired output behavior.



This data is used to fine-tune GPT-3 with supervised learning.



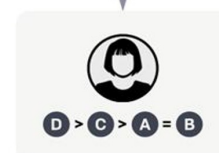
Step 2

Collect comparison data, and train a reward model.

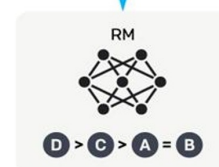
A prompt and several model outputs are sampled.



A labeler ranks the outputs from best to worst.



This data is used to train our reward model.



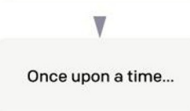
Step 3

Optimize a policy against the reward model using reinforcement learning.

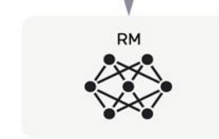
A new prompt is sampled from the dataset.



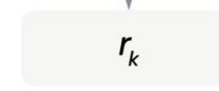
The policy generates an output.



The reward model calculates a reward for the output.



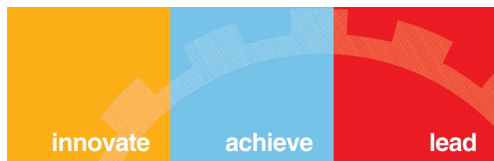
The reward is used to update the policy using PPO.



Instruction-Tuning

Reinforcement Learning from Human Feedback

Training language models to follow instructions with human feedback. Ouyang et al. 2022.



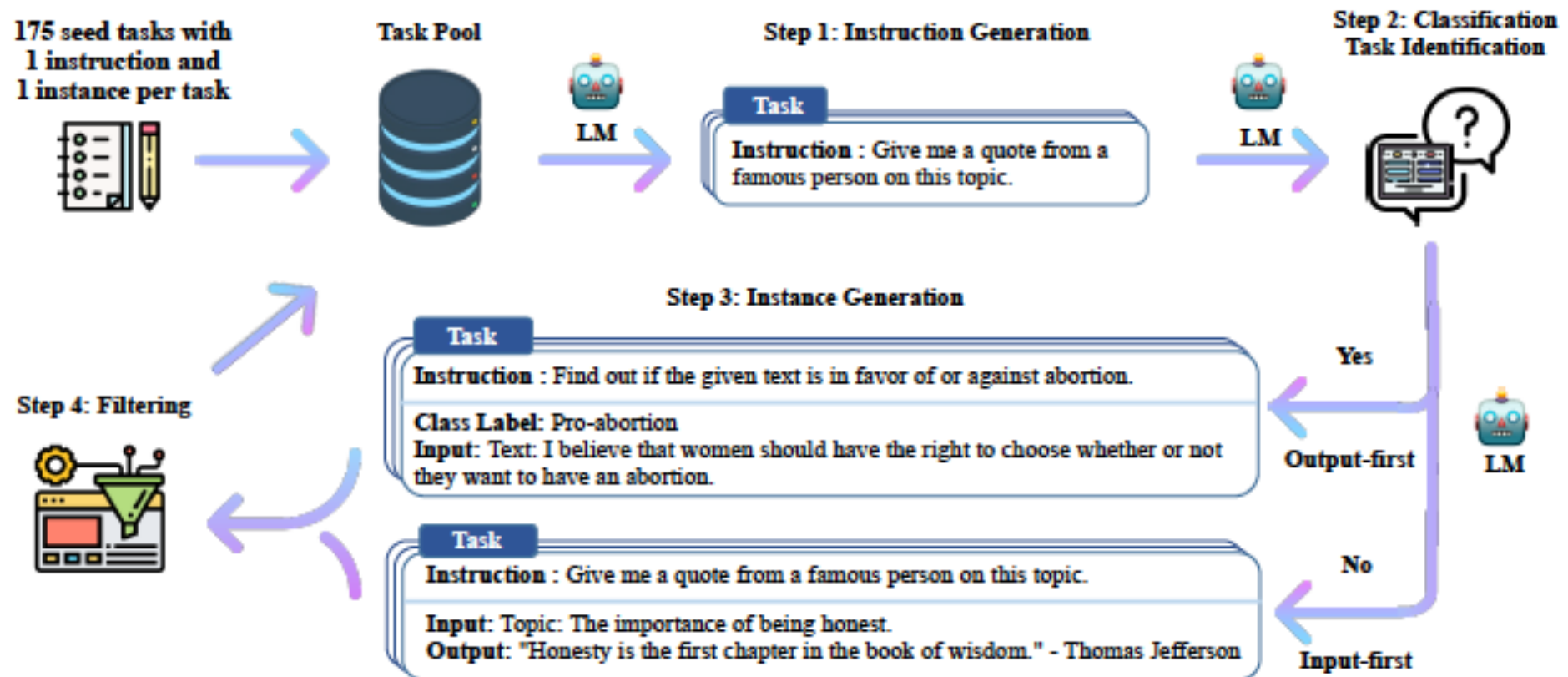
Self-Instruct: Aligning Language Models with Self-Generated Instructions-(*Wang et al., 2023*)

Instruction tuning requires large, diverse datasets of (instruction, output) pairs. Creating these datasets manually is expensive: human annotation at scale costs millions of dollars and months of time. Self-Instruct (Wang et al., 2023) addresses this bottleneck by using the LLM itself to generate its own training data.

Data Source	Characteristics
Human-Annotated Data (e.g., InstructGPT)	~13,000 human-written prompts + ranked responses. Cost: hundreds of annotator-hours. High quality but expensive and slow to scale.
Self-Instruct Data (e.g., Alpaca)	52,000 instruction-following examples generated by text-davinci-003. Cost: ~\$500 in API calls. Comparable quality for most tasks.

Human-written seed tasks are used to bootstrap **off-the-shelf language models** (e.g., GPT-3).

How does Self-Instruct work broadly?



Step 1: Instruction Generation

- The LM generates new task instructions based on existing tasks.

Step 2: Task Classification (Identification)

- The LM determines whether the instruction represents a
 - **Classification task** (Yes / No decision)
 - **Non-classification task**

Step 3: Instance Generation

The LM generates **input-output pairs** using one of two strategies:

Step 4: Filtering: Remove duplicates and low quality examples

Table6: Prompt used for classifying whether a task instruction



Can the following task be regarded as a classification task with finite output labels?

Task: Given my personality and the job, tell me if I would be suitable.

Is it classification? Yes

Task: Give me an example of a time when you had to use your sense of humor.

Is it classification? No

Task: Replace the placeholders in the given text with appropriate named entities.

Is it classification? No

Task: Fact checking - tell me if the statement is true, false, or unknown, based on your knowledge and common sense.

Is it classification? Yes

Task: Return the SSN number for the person.

Is it classification? No

Task: Detect if the Reddit thread contains hate speech.

Is it classification? Yes

Task: Analyze the sentences below to identify biases.

Is it classification? No

Task: Select the longest sentence in terms of the number of words in the paragraph, output the sentence index.

Is it classification? Yes

Task: Find out the toxic word or phrase in the sentence.

Is it classification? No

url = "https://aclanthology.org/2023.acl-long.754/"

Data Format Example



Instruction + Instance Input	Target Output	Validity
Instruction: Write a story with three characters: a person, an animal and an object. Input: Person: John, a programmer. Animal: A dog. Object: A laptop.	John was sitting on the floor in front of his laptop when he heard a knock at the door. He opened it to find a dog standing there. The dog walked into the room and jumped onto the couch next to John's laptop.	Instruction: ✓ Input: ✓ Output: ✓

GREAT:

Now we have a new **DATA SET!**

A "**Self-instruct generated**" Data Set!

Self-Instruct: Complete Data Generation Pipeline



Start: 100 Seed Tasks

Manually write 100 diverse task examples

Generate: Create More

Use GPT-3 to generate 10,000+ new instructions

Filter: Quality Assurance

Remove duplicates, low-quality, and biased examples

Fine-Tune: Train Your Model

Fine-tune your model on filtered dataset (10,000+ examples)

Result: 10,000+ diverse instruction examples with ~90-95% quality

Summary Table: Input-First vs. Output-First



Feature	Input-First	Output-First
Task Type	Creative / Open-ended	Classification (Finite labels)
Order	Generate Input → Answer it	Pick Label → Generate Input to match
Primary Goal	Natural logical flow	Preventing Class Imbalance/Bias

Why Output-First Matters for Classification

Suppose a sentiment classifier has 3 classes: Positive, Neutral, Negative. If we generate inputs first, the LLM's natural language bias will skew toward Positive examples (because positive language is more common in training data). Measured empirically: Input-First generates ~65% Positive, 25% Negative, 10% Neutral. Output-First generates ~33% each. A model trained on the skewed data achieves only 61% accuracy on a balanced test set; a model trained on the balanced Output-First data achieves 78%.



Experimental Results

- Use a GPT-3 (“davinci”) model to generate new instruction tasks, and fine-tune the GPT-3 model itself
- 175 seed tasks -> 52K instructions and 82K instances

statistic	
# of instructions	52,445
- # of classification instructions	11,584
- # of non-classification instructions	40,861
# of instances	82,439
- # of instances with empty input	35,878
ave. instruction length (in words)	15.9
ave. non-empty input length (in words)	12.7
ave. output length (in words)	18.9

Self-Instruct: Aligning Language Models with Self-Generated Instructions



Wang et al. 'Self-Instruct: Aligning Language Models with Self-Generated Instructions' (presented at ACL 2023) makes four primary contributions to the field:

1. Problem identification: articulates the instruction-following gap in pre-trained LLMs and frames it as a solvable data generation problem.
2. Pipeline design: proposes the four-step pipeline (generate → classify → instance generate → filter) that is now widely adopted in the field.
3. Empirical validation: demonstrates near-InstructGPT performance at a fraction of the cost on SUPERNI.
4. Data release: releases the 52K instruction dataset and code, enabling the broader community to fine-tune open-source models (leading directly to Alpaca, Vicuna, etc.).

Comparison of Using Different Instruction Tuning Datasets



Multiple instruction tuning datasets now exist, each with different trade-offs between cost, quality, diversity, and coverage

- There is not a single best instruction tuning dataset across all tasks
- Combining datasets results in the best overall performance

	MMLU (factuality)	GSM (reasoning)	BBH (reasoning)	TyDiQA (multilinguality)	Codex-Eval (coding)	AlpacaEval (open-ended)	Average
	EM (0-shot)	EM (8-shot, CoT)	EM (3-shot, CoT)	F1 (1-shot, GP)	P@10 (0-shot)	Win % vs Davinci-003	
Vanilla LLaMa 13B	42.3	14.5	39.3	43.2	28.6	-	-
+SuperNI	49.7	4.0	4.5	50.2	12.9	4.2	20.9
+CoT	44.2	40.0	41.9	47.8	23.7	6.0	33.9
+Flan V2	50.6	20.0	40.8	47.2	16.8	3.2	29.8
+Dolly	45.6	18.0	28.4	46.5	31.0	13.7	30.5
+Open Assistant 1	43.3	15.0	39.6	33.4	31.9	58.1	36.9
+Self-instruct	30.4	11.0	30.7	41.3	12.5	5.0	21.8
+Unnatural Instructions	46.4	8.0	33.7	40.9	23.9	8.4	26.9
+Alpaca	45.0	9.5	36.6	31.1	29.9	21.9	29.0
+Code-Alpaca	42.5	13.5	35.6	38.9	34.2	15.8	30.1
+GPT4-Alpaca	46.9	16.5	38.8	23.5	36.6	63.1	37.6
+Baize	43.7	10.0	38.7	33.6	28.7	21.9	29.4
+ShareGPT	49.3	27.0	40.4	30.5	34.1	70.5	42.0
+Human data mix.	50.2	38.5	39.6	47.0	25.0	35.0	39.2
+Human+GPT data mix.	49.3	40.5	43.3	45.6	35.9	56.5	45.2

How Far Can Camels Go? Exploring the State of Instruction Tuning on Open Resources, Yizhong Wang et. al., 2023

Practical Fine-Tuning: Making IT Affordable with PEFT



Full fine-tuning updates every parameter of a model. For a 7-billion-parameter model, this requires storing gradients for all 7B weights — demanding massive GPU memory and hours of compute.

PEFT (Parameter-Efficient Fine-Tuning) methods freeze the pre-trained weights and train only a small number of additional parameters, achieving near-full fine-tuning performance at a fraction of the cost.

Metric	GPU Type	Time/Model	Cost/Model	Storage	100 Models Cost
Full Fine-Tuning	H100 Cluster	7 days	\$5,000	7 GB	\$500,000
LoRA	Single GPU	1 day	\$500	10 MB	\$50,000
QLoRA	Consumer GPU	4 hours	\$100	10 MB	\$10,000

PEFT (Low-Rank Adaptation) enables efficient fine-tuning on commodity hardware, reducing costs by 98%+

What Comes Next: Lecture 7 - RLHF & PEFT Deep Dive



Reinforcement Learning from Human Feedback (RLHF)

Learn how to align models with human preferences using reward models and policy optimization

Direct Preference Optimization (DPO)

A simplified approach to alignment that bypasses the need for separate reward models

References & Further Reading



1. Wang, Y., et al. (2023). Self-Instruct: Aligning language models with self-generated instructions. ACL 2023.
2. Wei, J., et al. (2022). Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. arXiv:2201.11903
3. Yao, S., et al. (2023). Tree of Thoughts: Deliberate Problem Solving with Large Language Models. arXiv:2305.10601
4. Zhou, Y., et al. (2023). Large Language Models Are Human-Level Prompt Engineers. arXiv:2211.01910
5. Ouyang, L., et al. (2022). Training language models to follow instructions with human feedback. NeurIPS 2022.
6. Rafailov, R., et al. (2024). Direct Preference Optimization. arXiv:2305.18290
7. Dettmers, T., et al. (2023). QLoRA: Efficient finetuning of quantized LLMs. NeurIPS 2023.